Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №18

Дисциплина: Основы программирования на Python

Тема: Тестирование Python-приложений с использованием pytest и unittest.mock

Вариант 2: Интернет-магазин

Цель работы

Получить практические навыки автоматизированного тестирования Python-приложений. В процессе выполнения работы необходимо освоить организацию тестового проекта, написание unit-тестов, использование pytest, проверки assert и проверку исключений, фикстуры pytest, параметризованные тесты, изоляцию тестируемого кода от внешних зависимостей, использование unittest.mock, объекты Mock и MagicMock, функцию patch, тестирование классов и сервисного слоя, тестирование интерфейса, интеграционные тесты и анализ покрытия кода тестами.

Постановка индивидуального задания (Вариант 2. Интернет-магазин)

Разработать программу и создать для неё автоматизированный набор тестов. Приложение должно иметь три логических уровня: Repository, Service и Interface. В качестве предметной области используется интернет-магазин: товары (Product), заказы (Order) и их позиции (OrderItem). Требуется проверить создание товара, создание заказа, изменение количества товара на складе, расчёт стоимости заказа, а также невозможность заказать отсутствующий или имеющийся в недостаточном количестве товар, и обеспечить покрытие бизнес-логики unit-тестами с изоляцией от базы данных.

Краткие теоретические сведения

Автоматизированное тестирование — это проверка поведения программы кодом, который выполняется многократно и не требует ручных действий. Unit-тест проверяет изолированную единицу кода, чаще всего один метод или класс, и выполняется за доли секунды. Интеграционный тест проверяет совместную работу нескольких компонентов, например сервиса и реальной базы данных.

pytest — среда тестирования, в которой тестом считается функция с префиксом test_, а проверка выполняется оператором assert. Исключения проверяются контекстным менеджером pytest.raises(). Фикстуры объявляются декоратором pytest.fixture и передаются в тест как аргументы: они готовят данные и освобождают ресурсы после выполнения. Декоратор pytest.mark.parametrize позволяет запустить один тест с несколькими наборами входных данных, что заменяет копирование однотипных проверок.

Модуль unittest.mock предоставляет объекты-заглушки. Mock и MagicMock имитируют зависимости: первый требует явного описания атрибутов, второй позволяет обращаться к любым атрибутам и методам без ошибки. Методы assert_called_once(), assert_called_once_with() и call_count проверяют факт и параметры вызовов, атрибут return_value задаёт возвращаемое значение, а side_effect — последовательность значений или функцию, вызываемую при обращении. Функция patch временно подменяет объект или атрибут на время теста и восстанавливает исходное состояние после его завершения.

Покрытие кода (coverage) показывает долю выполненных строк и ветвей при прогоне тестов. Измерение выполняется плагином pytest-cov, который формирует отчёт по модулям и указывает непокрытые строки.

Архитектура тестируемого приложения

Приложение разделено на три уровня, что делает возможным независимое тестирование каждого из них.

Компонент	Назначение
src/models/product.py	Класс Product: название, цена, количество; валидация в __post_init__, свойство total_value (стоимость остатка)
src/models/order.py	Классы Order и OrderItem: позиции заказа, свойства total и items_count
src/repositories/sqlite_repository.py	SQLiteShopRepository: схема SQLite, CRUD товаров и заказов, транзакционное уменьшение остатков
src/services/shop_service.py	ShopService: валидация, создание товара/заказа, изменение количества, расчёт стоимости, иерархия исключений
src/services/cli.py	ConsoleUI: консольное меню, преобразование действий в вызовы сервиса
src/main.py	Точка входа и демонстрационное заполнение каталога

tests/conftest.py	Фикстуры repo, service, mock_repo, mock_service
tests/unit/, tests/integration/	Модульные и интеграционные тесты

Стратегия тестирования

Тесты организованы по уровням, образуя пирамиду: наибольшее число проверок приходится на модели и сервис, меньшее — на интеграционные сценарии с реальной базой. Такое распределение обеспечивает быстрые прогоны и точную локализацию ошибок.

Группа тестов	Зависимости	Что проверяется
tests/unit/test_models.py	нет	Валидация товара (пустое имя, отрицательные цена/количество), стоимость остатка, стоимость и количество заказа
tests/unit/test_shop_service.py	MagicMock	Создание товара и заказа, изменение количества, расчёт стоимости, отсутствие товара, недостаток остатка, пустой заказ, некорректное количество, параметризованные проверки
tests/unit/test_cli.py	Mock сервиса	Формирование текстовых ответов, добавление товара, сбор позиций заказа, обработка ошибок сервиса
tests/unit/test_main.py	MagicMock	Демонстрационное заполнение каталога тремя товарами
tests/integration/test_repository.py	реальная SQLite	Полный CRUD товара, изменение количества, создание заказа с уменьшением остатков, откат транзакции при недостатке остатка

Изоляция бизнес-логики с помощью Mock

Сервис принимает репозиторий в конструкторе, поэтому в модульных тестах вместо реальной базы подставляется MagicMock. Возвращаемые значения настраиваются через

return_value, факт вызова проверяется методами assert_called_once() и assert_called_once_with(), a side_effect задаёт функцию или последовательность возвращаемых значений.

Листинг 18.1 — Фикстура Mock-репозитория (tests/conftest.py)

```
@pytest.fixture()
def mock_repo() -> MagicMock:
    return MagicMock()

@pytest.fixture()
def mock_service(mock_repo: MagicMock) -> ShopService:
    return ShopService(mock_repo)
```

Листинг 18.2 — Unit-тест создания заказа (tests/unit/test_shop_service.py)

```
def test_create_order_success(mock_repo: MagicMock):
    mock_repo.get_product.return_value = Product(id=1, name="Мышь", price=1500,
quantity=5)
    mock_repo.save_order.side_effect = lambda customer, items: Order(
        id=1, customer=customer, items=list(items)
    )
    service = ShopService(mock_repo)

    order = service.create_order("Иванов", [(1, 2)])

    assert order.total == 3000
    mock_repo.save_order.assert_called_once()
    customer, items = mock_repo.save_order.call_args.args
    assert customer == "Иванов"
    assert items[0].quantity == 2
```

Параметризованные тесты и проверка исключений

Параметризация заменяет несколько почти одинаковых тестов одним: декоратор parametrize задаёт наборы входных значений и ожидаемый результат, а pytest формирует отдельный запуск для каждого набора. В работе параметризованы проверки стоимости остатка товара и проверки некорректного количества в заказе, где ожидается исключение ValidationError.

Листинг 18.3 — Параметризованные тесты (tests/unit/test_models.py, test_shop_service.py)

```
@pytest.mark.parametrize(
    "price,quantity,expected",
    [
        (0, 0, 0),
        (100, 2, 200),
        (99.9, 3, 299.7),
    ],
)
def test_product_total_value_parametrized(price, quantity, expected):
    product = Product(id=1, name="Товар", price=price, quantity=quantity)
    assert product.total_value == expected

@pytest.mark.parametrize("quantity", [0, -1, -5])
def test_create_order_rejects_non_positive_quantity(mock_repo: MagicMock,
quantity):
```

```
    service = ShopService(mock_repo)
    with pytest.raises(ValidationError):
        service.create_order("Иванов", [(1, quantity)])
```

Использование patch и side_effect

Функция patch применяется в тестах консольного интерфейса для подмены встроенной функции input(): это позволяет проверить, какие аргументы получает сервис при добавлении товара или создании заказа. Атрибут side_effect задаёт последовательность значений, возвращаемых для последовательных вызовов одной функции.

Листинг 18.4 — Использование patch (tests/unit/test_cli.py)

```
def test_cli_add_product_with_patch():
    service = MagicMock()
    service.create_product.return_value = MagicMock(id=1, name="Мышь")
    ui = ConsoleUI(service)

    with patch("builtins.input", side_effect=["Мышь", "1500", "3"]):
        ui._add_product()

    service.create_product.assert_called_once_with("Мышь", 1500.0, 3)
```

Интеграционные тесты

Интеграционные тесты работают с настоящей базой SQLite в памяти: фикстура repo создаёт репозиторий, а фикстура service — сервис поверх него. Каждый тест получает чистую базу. Такие тесты проверяют не только логику сервиса, но и корректность SQL-запросов, схемы, сохранения состояния и транзакционного отката.

Листинг 18.5 — Фикстуры интеграционных тестов (tests/conftest.py)

```
@pytest.fixture()
def repo() -> SQLiteShopRepository:
    return SQLiteShopRepository(":memory:")

@pytest.fixture()
def service(repo: SQLiteShopRepository) -> ShopService:
    return ShopService(repo)
```

Листинг 18.6 — Интеграционный тест заказа с уменьшением остатков (tests/integration/test_repository.py)

```
def test_create_order_decrements_stock(real_service: ShopService):
    keyboard = real_service.create_product("Клавиатура", 3500, 5)
    mouse = real_service.create_product("Мышь", 1500, 10)

    order = real_service.create_order("Иванов", [(keyboard.id, 2), (mouse.id, 1)])

    assert order.total == 8500
    assert real_service.get_product(keyboard.id).quantity == 3
    assert real_service.get_product(mouse.id).quantity == 9
```

```
stored = real_service.get_order(order.id)
assert stored.total == 8500
assert len(stored.items) == 2
```

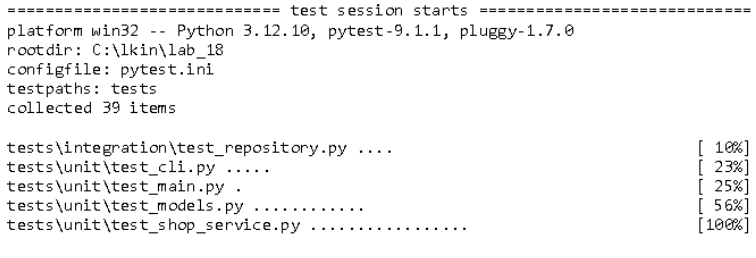
Результаты выполнения тестов

Тесты запускаются командой pytest из корня проекта. В файле pytest.ini заданы каталог tests и строгий режим маркеров, что позволяет выборочно запускать нужные группы, например pytest -m unit.

Файл тестов	Тип	Результат
tests/unit/test_models.py	модульные	пройдены, проверены свойства модели и граничные случаи
tests/unit/test_shop_service.py	модульные с Mock	пройдены, включая параметризованные наборы данных
tests/unit/test_cli.py	модульные с Mock	пройдены, ответы интерфейса соответствуют ожидаемым
tests/unit/test_main.py	модульные	пройден, демонстрационное заполнение выполняется без ошибок
tests/integration/test_repository.py	интеграционные	пройдены на реальной базе SQLite в памяти

Сводный отчёт pytest показывает, что все 39 тестов завершились успешно: 37 unit-тестов и 2 интеграционных, сообщений об ошибках и предупреждениях нет.

Рисунок 18.1 — Результат выполнения тестов (pytest -v)



Анализ покрытия и обнаруженные ошибки

Покрытие измерялось плагином pytest-cov командой pytest --cov=src --cov-report=term-missing. Итоговое покрытие проекта составило 80 процентов. Полнее всего покрыты модели, сервисный слой и исключения: проверены как успешные сценарии, так и все ветви обработки ошибок, включая отсутствие товара, недостаток остатка и некорректное количество в заказе. Меньшее покрытие имеют ветви консольного интерфейса и некоторые служебные функции репозитория.

Рисунок 18.2 — Отчёт о покрытии кода (pytest --cov=src)

```
Name                                                 Stmts   Miss  Cover   Missing
----------------------------------------------------------------------------------
C:\lkin\lab_18\src\__init__.py                           0      0   100%
C:\lkin\lab_18\src\errors.py                             4      0   100%
C:\lkin\lab_18\src\main.py                              15      5    67%   16-19, 23
C:\lkin\lab_18\src\models\__init__.py                    0      0   100%
C:\lkin\lab_18\src\models\order.py                      25      0   100%
C:\lkin\lab_18\src\models\product.py                    20      0   100%
C:\lkin\lab_18\src\repositories\__init__.py              0      0   100%
C:\lkin\lab_18\src\repositories\sqlite_repository.py    75     10    87%   78-81, 98, 109, 112, 146, 150-152, 162
C:\lkin\lab_18\src\services\__init__.py                  0      0   100%
C:\lkin\lab_18\src\services\cli.py                      72     36    50%   13-33, 37-43, 50-51, 67, 83-88
C:\lkin\lab_18\src\services\shop_service.py             64      3    95%   29, 50, 95
----------------------------------------------------------------------------------
TOTAL                                                  275     54    80%
```

В процессе написания тестов были обнаружены и устранены следующие проблемы. Во-первых, операция изменения количества допускала нулевое изменение, что не имеет смысла с точки зрения бизнес-логики; добавлена проверка, возбуждающая ValidationError, и соответствующий тест. Во-вторых, при создании заказа сервис контролировал остаток только на уровне сервиса, а репозиторий дополнительно проверял остаток внутри транзакции и выполнял её откат, если товара не хватало; интеграционный тест подтверждает, что при ошибке не создаётся ни заказа, ни изменения остатков. В-третьих, валидация модели допускала отрицательную цену и количество; в конструктор добавлены проверки, покрытые граничными тестами.

Вывод

В ходе выполнения лабораторной работы освоено автоматизированное тестирование Python-приложений. Создан тестовый проект, разделённый на модульные и интеграционные тесты, использованы фикстуры, параметризация и проверка исключений. Изоляция бизнес-логики достигнута заменой репозитория объектом MagicMock, а функция patch позволила проверить

поведение консольного интерфейса при подмене ввода. Интеграционные тесты на реальной базе SQLite подтвердили корректность запросов, сохранения состояния и транзакционного отката. Набор тестов выявил несколько ошибок в логике валидации и заказов, которые были исправлены, а анализ покрытия показал, что наиболее важные ветви сервисного слоя проверены полностью.